

PhishGuard: A Multi-Model Machine Learning Framework for Phishing Website Detection Using Mathematical Feature Engineering and Stacking Ensemble Classification

Abhijit Satyaki^{a,*}, Surajit Das^a, Bikash Majhi^a, Prof. Subrata Kumar Mondal^a

^aDepartment of Information Technology, Jalpaiguri Government Engineering College, Jalpaiguri, West Bengal, India

Abstract

Phishing attacks remain one of the most pervasive threats in cybersecurity, with attackers employing increasingly sophisticated techniques to deceive users through fraudulent websites. This paper presents PhishGuard, a comprehensive machine learning framework for phishing website detection that integrates four novel mathematical models—Damerau-Levenshtein typosquatting analysis, character-level n-gram language modeling, Unicode homoglyph detection, and Shannon entropy analysis—with an ensemble of eleven supervised classifiers orchestrated through a stacking meta-learner. The framework operates exclusively on URL-structural and mathematical features that are deterministically computable from the URL string itself, eliminating dependencies on external services or content rendering. A dual feature selection strategy combining ANOVA F-test univariate filtering with Random Forest model-based importance ranking identifies the optimal 50-feature subset from an initial pool of 70 candidate features through set intersection. Evaluated on a dataset of 50,000 labeled URLs, the proposed stacking ensemble achieves 99.07% accuracy, 99.33% precision, 98.80% recall, and 0.9996 AUC-ROC, while XGBoost attains the highest individual accuracy of 99.15%. An Isolation Forest anomaly detector provides complementary unsupervised coverage for zero-day phishing patterns absent from training data. Experimental results across all twelve models demonstrate that mathematical feature engineering significantly improves detection performance over purely structural URL features, establishing PhishGuard as a robust, real-time capable phishing detection system.

Keywords: Phishing detection, Machine learning, Ensemble learning, Stacking classifier, Typosquatting, N-gram language model, Homoglyph detection, Shannon entropy, Feature engineering, Anomaly detection, URL analysis, Cybersecurity

1. Introduction

Phishing attacks constitute a critical and growing threat vector in the modern cybersecurity landscape, accounting for over 36% of all data breaches according to the 2024 Verizon Data Breach Investigations Report Verizon (2024). These attacks exploit human trust by presenting fraudulent websites that visually impersonate legitimate services, deceiving users into divulging sensitive credentials, financial information, and personal data. The Anti-Phishing Working Group (APWG) reported over 4.7 million phishing attacks in 2023 alone Anti-Phishing Working Group (2024), marking a continued escalation in both volume and sophistication. Traditional defense mechanisms—including static blacklists, heuristic rules, and signature-based approaches—suffer from inherent limitations in detecting novel phishing campaigns, as attackers rapidly rotate domains, employ domain generation algorithms (DGAs), and leverage internationalized domain names to evade static filters Jain and Gupta (2022).

Machine learning (ML) approaches have emerged as the predominant paradigm for automated phishing detection, offering the ability to generalize from labeled training examples

to previously unseen phishing URLs Mohammad et al. (2015); Marchal et al. (2014). However, existing ML-based systems face several persistent challenges. First, many rely on content-based features extracted from the rendered HTML of target pages Zhang et al. (2007); Xiang et al. (2011), introducing latency from HTTP requests and vulnerability to cloaking techniques where phishing pages serve benign content to automated scanners while displaying malicious content to human visitors Invernizzi et al. (2016). Second, reliance on external reputation services (WHOIS lookups, DNS records, PageRank queries) creates dependencies on third-party infrastructure that may be rate-limited, unavailable, or manipulated Le et al. (2018a). Third, the feature engineering process in most existing works treats URL features as purely structural counts (character frequencies, length metrics), neglecting the rich information-theoretic and linguistic signals embedded within domain names Aljofey et al. (2020a).

This paper presents PhishGuard, a machine learning framework that addresses these limitations through three principal contributions:

- **Mathematical Feature Engineering:** We introduce four specialized mathematical models that extract semantic and statistical signals from URL strings. The Damerau-Levenshtein typosquatting analyzer quantifies domain similarity to known legitimate brands. A character-level trigram language model computes domain perplexity to detect algorithmically gen-

*Corresponding author

Email addresses: abhijitsatyaki29@gmail.com (Abhijit Satyaki), sura.jit202425@gmail.com (Surajit Das), bikashmajhi837@gmail.com (Bikash Majhi)

erated domains. A Unicode homoglyph detector identifies internationalized domain name (IDN) attacks using confusable character mappings. A Shannon entropy analyzer flags domains with anomalous randomness characteristic of DGA output.

- **Comprehensive Ensemble Architecture:** We train and evaluate eleven heterogeneous classifiers spanning decision trees, kernel methods, neural networks, nearest-neighbor models, probabilistic classifiers, and gradient-boosted forests, culminating in a stacking ensemble that combines the five strongest base learners under a logistic regression meta-learner with five-fold cross-validated predictions.
- **Inference-Time Feature Parity:** The framework is architecturally designed so that every feature used during training is deterministically computable from the URL string at inference time, without requiring HTTP requests, DNS resolution, or external API calls. This ensures zero feature drift between training and deployment, enabling real-time prediction with sub-second latency.

The remainder of this paper is organized as follows. Section 2 reviews related work in phishing detection and machine learning-based URL analysis. Section 3 presents the proposed framework in detail, including mathematical models, classification architecture, feature engineering pipeline, and anomaly detection. Section 4 reports experimental results and comparative analysis. Section 5 discusses findings, limitations, and practical implications. Section 6 concludes the paper with directions for future work.

2. Related Work

2.1. Traditional Phishing Detection Approaches

Early phishing detection systems relied primarily on blacklist-based methods, maintaining databases of known phishing URLs and matching incoming requests against these lists Ma et al. (2009b). Google Safe Browsing Google (2024) and PhishTank PhishTank (2024) exemplify this approach, offering broad coverage but suffering from an inherent inability to detect zero-day phishing URLs not yet catalogued. Blacklist update latency—often measured in hours—creates a critical vulnerability window during which newly launched phishing campaigns operate undetected Chou et al. (2004).

Heuristic approaches improved upon blacklists by defining rule-based features such as the presence of IP addresses in URLs, suspicious TLD usage, and URL length thresholds Khonji et al. (2013). While these heuristics capture common phishing patterns, they are brittle against adaptive adversaries who engineer URLs specifically to pass known rule checks Sahoo et al. (2017).

2.2. Machine Learning-Based URL Classification

The application of machine learning to phishing URL classification has been extensively studied. Ma et al. (2009a) demonstrated that lexical features of URLs—including character distributions, token lengths, and special character frequencies—provide sufficient discriminative power for binary classification

without requiring page content. Subsequent work by Le et al. (2018b) introduced URL embedding approaches using recurrent neural networks, while Sahingoz et al. (2019) conducted a comprehensive comparison of seven machine learning classifiers on URL-based features, reporting Random Forest as the top performer with 97.98% accuracy.

Ensemble methods have shown particular promise in this domain. Rao and Pais (2019) employed a voting ensemble of Random Forest, Gradient Boosting, and Extra Trees classifiers, achieving 98.4% accuracy on the PhishTank dataset. Chiew et al. (2018) proposed a hybrid ensemble combining content-based and URL-based features through a cascade architecture. More recently, gradient-boosted tree methods—XGBoost Chen and Guestrin (2016) and LightGBM Ke et al. (2017)—have become the de facto standard for tabular phishing classification, consistently outperforming deep learning alternatives on structured feature sets Shwartz-Ziv and Armon (2022).

2.3. Feature Engineering for URL Analysis

Feature engineering remains the single most impactful factor in phishing detection performance Verma and Dyer (2015). Muhammad et al. (2023) categorized URL features into lexical, host-based, content-based, and external-service features, demonstrating that lexical features alone achieve 94–96% accuracy. Aljofey et al. (2020b) augmented structural features with information-theoretic measures including Shannon entropy and Kolmogorov complexity estimates, reporting a 2.3% accuracy improvement.

Typosquatting detection has been addressed as a standalone problem by Szurdi et al. (2014), who applied edit distance metrics to identify domain names that are deliberate misspellings of popular brands. However, their work did not integrate typosquatting scores as features within a broader ML classification pipeline. Similarly, Unicode-based attacks through internationalized domain names (IDN homograph attacks) have been studied by Holgers et al. (2006) in isolation, without incorporation into feature-based classifiers.

2.4. Anomaly Detection in Cybersecurity

Unsupervised anomaly detection complements supervised classification by identifying statistical outliers that deviate from learned patterns of normal behavior Chandola et al. (2009). Isolation Forest Liu et al. (2008), based on recursive random partitioning, has proven effective for detecting network intrusions and novel malware Ahmed et al. (2016). In the phishing domain, anomaly detection addresses the critical limitation of supervised models: their inability to detect phishing patterns fundamentally different from those observed during training Buber et al. (2017).

The present work differs from prior literature in several key aspects: (i) it is the first to integrate four distinct mathematical models—typosquatting scoring, n-gram language modeling, homoglyph detection, and entropy analysis—as engineered features within a unified classification pipeline; (ii) it employs a dual feature selection strategy combining statistical and model-based methods through set intersection; (iii) it achieves strict

inference-time feature parity by design, using only URL-computable features; and (iv) it combines supervised stacking ensemble classification with unsupervised Isolation Forest anomaly detection in a complementary architecture.

3. Proposed Framework

The PhishGuard framework consists of four integrated sub-systems: (A) mathematical models for feature engineering, (B) a multi-classifier ensemble with stacking meta-learner, (C) a feature engineering and selection pipeline, and (D) an Isolation Forest anomaly detector. Fig. 1 illustrates the stacking ensemble architecture, and Fig. 8 depicts the complete feature engineering pipeline.



Figure 1—Architecture of the stacking ensemble classifier with five base learners and logistic regression meta-learner.

3.1. Mathematical Models for Feature Engineering

The framework introduces four mathematical models that extract semantic and statistical signals beyond traditional structural URL features. Each model produces a normalized score in the range $[0, 1]$, where 0 indicates a benign signal and 1 indicates a maximally suspicious signal.

3.1.1. Damerau-Levenshtein Typosquatting Analysis

Typosquatting—the registration of domain names that are deliberate misspellings of popular brands—is a prevalent phishing technique. We detect typosquatting by computing the Damerau-Levenshtein (DL) distance Damerau (1964) between the target domain’s second-level domain (SLD) and each entry in a reference list of 1,000 legitimate brand domains (Alexa Top 1K).

The DL distance extends classical Levenshtein distance by adding transposition as a permitted edit operation, making it

more suitable for modeling human typing errors. For strings s_1 and s_2 of lengths m and n respectively, the DL distance is computed via the recurrence:

$$d_{i,j} = \min \begin{cases} d_{i-1,j} + 1 & \text{(deletion)} \\ d_{i,j-1} + 1 & \text{(insertion)} \\ d_{i-1,j-1} + c(s_1[i], s_2[j]) & \text{(substitution)} \\ d_{i-2,j-2} + c(s_1[i], s_2[j]) & \text{if } s_1[i] = s_2[j-1] \\ & \text{and } s_1[i-1] = s_2[j] \end{cases} \quad (1)$$

(transposition)

where $c(a, b) = 0$ if $a = b$ and $c(a, b) = 1$ otherwise. The base cases are $d_{i,0} = i$ and $d_{0,j} = j$.

The typosquatting score is then computed as:

$$\text{TypoScore}(d) = \begin{cases} 0.00 & \text{if } d^* = 0 \text{ (exact match)} \\ 0.95 & \text{if } d^* = 1 \text{ (critical squatting)} \\ 0.60 & \text{if } d^* = 2 \text{ (likely squatting)} \\ 0.25 & \text{if } d^* = 3 \text{ and } |s_1| \leq 8 \\ 0.00 & \text{otherwise} \end{cases} \quad (2)$$

where $d^* = \min_{b \in \mathcal{B}} \text{DL}(s_1, b)$ is the minimum distance to any brand b in the reference set $\mathcal{B}$.

An early-termination optimization skips brand comparisons when $|m - n| > 4$, as such pairs cannot have small edit distances. Fig. 2 presents a heatmap of DL distances between example domains and known brands.



Figure 2—Heatmap of Damerau-Levenshtein distances between sample domains and known brand names, illustrating typosquatting proximity.

3.1.2. Character N-gram Language Model

Domain generation algorithms (DGAs) produce pseudo-random domain names that appear as gibberish to human readers. We detect such domains using a character-level trigram ($n = 3$) language model trained on legitimate domain names from the Alexa Top 1K and a curated list of 65 major brand names.

The model estimates the probability of a domain string $d = c_1c_2 \dots c_L$ under the learned character distribution:

$$P(d) = \prod_{i=1}^{L+1} P(c_i | c_{i-n+1} \dots c_{i-1}) \quad (3)$$

where the domain is padded with $n - 1$ start symbols and a terminal symbol. Each conditional probability is estimated with Laplace (add-one) smoothing:

$$P(c_i | h) = \frac{\text{count}(h \cdot c_i) + 1}{\text{count}(h) + |V| + 1} \quad (4)$$

where h is the $(n - 1)$ -character context (history), $\text{count}(\cdot)$ denotes frequency in the training corpus, and $|V|$ is the vocabulary size.

The normalized log-probability per character is:

$$\ell(d) = \frac{1}{L+1} \sum_{i=1}^{L+1} \log P(c_i | h_i) \quad (5)$$

Perplexity—the exponential of negative log-probability—provides the final suspicion metric:

$$\text{PP}(d) = \exp(-\ell(d)) \quad (6)$$

Lower perplexity indicates a domain consistent with legitimate naming conventions (e.g., “google” $\rightarrow \text{PP} \approx 3.2$), while higher perplexity flags gibberish domains (e.g., “xj8kkl2” $\rightarrow \text{PP} \approx 45.7$). The perplexity is mapped to a normalized score:

$$\text{PerplexityScore}(d) = \begin{cases} 0.0 & \text{if } \text{PP}(d) < 8.0 \\ 0.2 & \text{if } 8.0 \leq \text{PP}(d) < 15.0 \\ 0.5 & \text{if } 15.0 \leq \text{PP}(d) < 30.0 \\ 0.75 & \text{if } 30.0 \leq \text{PP}(d) < 60.0 \\ 1.0 & \text{if } \text{PP}(d) \geq 60.0 \end{cases} \quad (7)$$

Fig. 3 shows perplexity distributions for legitimate versus DGA-generated domains.

paper_figures/fig8_ngram_perplexity.png

Figure 3—N-gram perplexity comparison between legitimate domains and DGA-generated domain names.

3.1.3. Unicode Homoglyph Detection

Internationalized Domain Name (IDN) homograph attacks exploit visually similar Unicode characters to impersonate legitimate domains. For example, Cyrillic “p” (U+0440) is visually indistinguishable from Latin “p” (U+0070), enabling an attacker to register a convincing phishing domain such as “paypal.com” with mixed scripts.

Our homoglyph detector maintains a mapping $\mathcal{H} : \mathcal{U} \rightarrow \mathcal{A}$ from Unicode code points to their ASCII visual equivalents, covering Cyrillic-to-Latin mappings (a, e, o, p, c, y, x, i, s), Greek-to-Latin mappings (a, o, p, v, t), and fullwidth form normalizations.

For a domain string $d = u_1u_2 \dots u_L$, the homoglyph score is:

$$\text{HomoglyphScore}(d) = \begin{cases} 1.0 & \text{if } \exists i : u_i \in \mathcal{H} \setminus \mathcal{N} \\ 0.8 & \text{if } d \text{ contains non-ASCII} \\ & \text{characters not in } \mathcal{H} \\ 0.0 & \text{otherwise} \end{cases} \quad (8)$$

where $\mathcal{N} = \{0, 1, 5, 6, 8\}$ is the set of numeric characters excluded from flagging (as digits commonly appear in URLs). The detector also produces a normalized ASCII version of the domain by replacing all mapped characters, enabling downstream comparison against brand dictionaries.

3.1.4. Shannon Entropy Analysis

Shannon entropy Shannon (1948) measures the information density of a string, providing a statistical signature of randomness. DGA-generated domains exhibit characteristically

high entropy due to their pseudo-random character composition, while legitimate domains tend toward lower entropy with natural language patterns.

For a domain string d with character frequency distribution $\{p(c)\}_{c \in \Sigma}$:

$$H(d) = - \sum_{c \in \Sigma} p(c) \log_2 p(c) \quad (9)$$

where $p(c) = \frac{\text{count}(c,d)}{|d|}$ is the relative frequency of character c in d .

Empirical analysis of our training corpus establishes the following entropy ranges: legitimate domains typically fall within $H \in [2.5, 3.5]$ bits, while DGA domains concentrate in $H \in [3.8, 4.5]$ bits. The entropy is mapped to a normalized detection score:

$$\text{EntropyScore}(d) = \begin{cases} 0.0 & \text{if } H(d) < 3.0 \\ 0.2 & \text{if } 3.0 \leq H(d) < 3.5 \\ 0.5 & \text{if } 3.5 \leq H(d) < 3.8 \\ 0.75 & \text{if } 3.8 \leq H(d) < 4.2 \\ 1.0 & \text{if } H(d) \geq 4.2 \end{cases} \quad (10)$$

Fig. 4 presents the entropy distribution comparison between legitimate and phishing domains in our dataset. Fig. 5 provides illustrative examples of mathematical model outputs across representative domain categories.

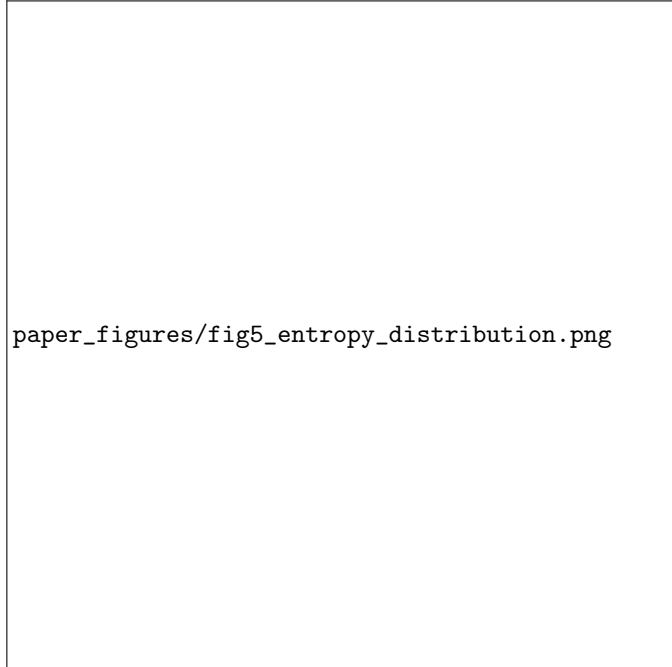


Figure 4—Shannon entropy distributions for legitimate versus phishing domain names, showing the separation boundary at

$$H \approx 3.5 \text{ bits.}$$

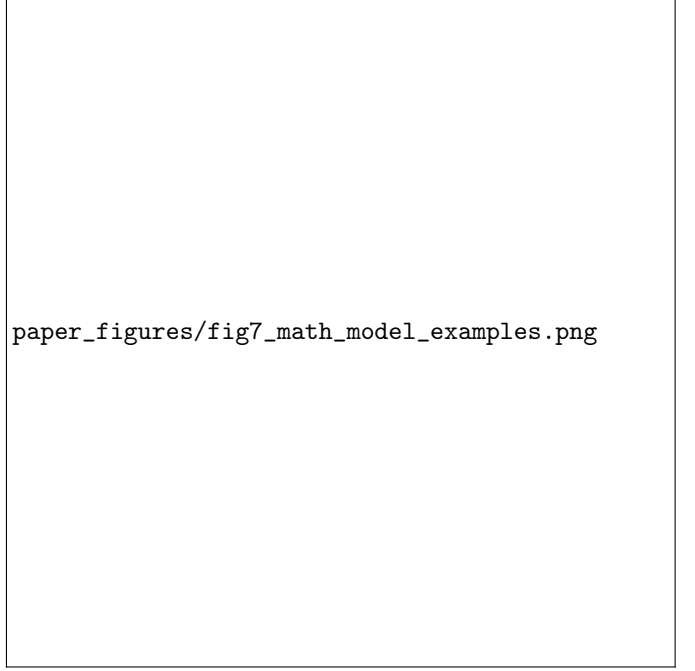


Figure 5—Mathematical model output examples across domain categories: legitimate brands, typosquatting variants, DGA-generated, and homoglyph attacks.

3.2. Classification Architecture

The classification subsystem employs eleven heterogeneous supervised learning algorithms, selected to maximize diversity in inductive bias, decision boundary geometry, and sensitivity to feature interactions. Table 1 summarizes all classifier configurations.

3.2.1. Stacking Ensemble Meta-Learner

The stacking ensemble Wolpert (1992) selects the five highest-performing base learners—Random Forest, Gradient Boosting, Extra Trees, XGBoost, and LightGBM—as level-0 estimators. A Logistic Regression model serves as the level-1 meta-learner, trained on five-fold cross-validated predictions from the base learners to mitigate overfitting.

Formally, let $f_1, f_2, \dots, f_5$ denote the base learners. For a training sample $\mathbf{x}_i$, the cross-validated prediction of the k -th base learner is $\hat{z}_{i,k} = f_k^{(-i)}(\mathbf{x}_i)$, where $f_k^{(-i)}$ denotes the model trained on all folds excluding the one containing $\mathbf{x}_i$. The meta-feature matrix $\mathbf{Z} = [\hat{z}_{i,k}]_{N \times 5}$ is then used to train the meta-learner:

$$g(\mathbf{z}_i) = \sigma(\mathbf{w}^T \mathbf{z}_i + b) \quad (11)$$

where σ is the logistic sigmoid function, $\mathbf{w} \in \mathbb{R}^5$ are the learned base-learner weights, and b is the bias term. The final ensemble prediction is:

$$\hat{y} = \begin{cases} 1 \text{ (phishing)} & \text{if } g(\mathbf{z}) \geq 0.5 \\ 0 \text{ (legitimate)} & \text{otherwise} \end{cases} \quad (12)$$

Table 1: Classifier configurations and hyperparameters used in the PhishGuard framework

Classifier	Key Hyperparameters
Random Forest (RF)	$T = 200$ trees, max depth = 15
Gradient Boosting (GB)	$T = 150$ trees, $\eta = 0.1$, max depth = 8
Extra Trees (ET)	$T = 200$ trees, max depth = 15
XGBoost (XGB)	$T = 150$ trees, $\eta = 0.1$, max depth = 8
LightGBM (LGBM)	$T = 150$ trees, $\eta = 0.1$, max depth = 8
Support Vector Machine (SVM)	RBF kernel, $C = 1.0$, $\gamma = \text{scale}$
Multi-Layer Perceptron (MLP)	Layers: 128-64-32, $\alpha = 0.001$, max iter = 500
Logistic Regression (LR)	$C = 1.0$, max iter = 1000
K-Nearest Neighbors (KNN)	$k = 5$, Minkowski distance
Decision Tree (DT)	Max depth = 10
Gaussian Naive Bayes (GNB)	Default priors

This two-level architecture enables the meta-learner to learn the optimal weighting and complementary strengths of each base classifier, correcting for individual model biases.

3.2.2. Individual Classifier Training

Each of the eleven base classifiers is trained on the standardized feature matrix using five-fold stratified cross-validation to estimate generalization performance. The cross-validation AUC-ROC serves as the primary model selection criterion. All classifiers are trained with probability calibration enabled (where applicable) to produce meaningful confidence estimates for the stacking meta-learner.

3.3. Feature Engineering and Selection Pipeline

The feature engineering pipeline transforms a raw URL string into a 50-dimensional standardized feature vector through four stages: structural extraction, mathematical augmentation, interaction engineering, and dual-method selection.

3.3.1. Structural Feature Extraction

The first stage extracts 57 structural features from the URL through lexical parsing. These include:

- **Length features:** URL length, hostname length, word lengths (shortest, longest, average for URL/host/path)
- **Character frequency features:** counts of dots, hyphens, at-signs, question marks, ampersands, equals signs, underscores, tildes, percent signs, slashes, colons, semi-colons, dollar signs, spaces, double slashes
- **Ratio features:** digit-to-character ratios for URL and hostname, with UUID-aware normalization that excludes legitimate UUID path segments from the digit count
- **Binary indicator features:** IP address as hostname, punycode encoding, non-standard port, TLD in path/subdomain, prefix-suffix hyphenation, URL shortening service, path extension presence

- **Brand proximity features:** domain-in-brand, brand-in-subdomain, brand-in-path, phishing keyword count
- **TLD analysis:** suspicious TLD indicator for high-abuse TLDs (.tk, .ml, .ga, .cf, .gq, .xyz, .top, .zip, .mov, and 13 others)

Unreliable Feature Exclusion: A critical design decision is the explicit exclusion of 28 features that cannot be reliably computed at inference time. These include external service features (PageRank, domain age, WHOIS data, DNS records, web traffic), HTML content features (hyperlink ratios, CSS counts, form elements, iframe presence), and misleading count features (nb_com, nb_www) that produce identical values for legitimate and phishing .com/.www domains. This exclusion ensures zero feature drift between training and deployment.

3.3.2. Mathematical Feature Augmentation

The second stage augments the structural features with four mathematical model scores (typosquatting_score, domain_perplexity_score, homoglyph_score, domain_entropy_score) as described in Section 3A, plus a UUID-in-path binary indicator. This brings the candidate feature count to approximately 62.

3.3.3. Interaction Feature Engineering

The third stage generates five multiplicative and ratio interaction features that capture cross-feature relationships:

$$\text{length_url} \times \text{nb_dots}, \quad \frac{\text{length_url}}{\text{nb_dots} + \epsilon}, \quad \frac{\text{length_url}}{\text{nb_slash} + \epsilon} \quad (13)$$

$$\text{length_url} \times \text{nb_hyphens}, \quad \text{nb_dots} \times \text{nb_hyphens} \quad (14)$$

where $\epsilon = 10^{-6}$ prevents division by zero. These interactions capture patterns such as “long URLs with many dots” (characteristic of deep subdomain phishing) that individual features cannot represent.

3.3.4. Dual Feature Selection Strategy

The fourth stage applies a novel dual selection method that intersects two independent feature ranking approaches to identify robust features:

Method 1—ANOVA F-test (univariate statistical filter): Computes the F-statistic between each feature and the target variable, selecting the top $k = 50$ features by statistical significance:

$$F_j = \frac{SS_{\text{between},j}/(K-1)}{SS_{\text{within},j}/(N-K)} \quad (15)$$

where $SS_{\text{between},j}$ and $SS_{\text{within},j}$ are the between-class and within-class sum of squares for feature j , K is the number of classes, and N is the number of samples.

Method 2—Random Forest Importance (model-based filter): Trains a Random Forest with 50 estimators and selects features whose Gini importance exceeds a data-driven threshold via `SelectFromModel`.

The final feature set is computed as:

$$\mathcal{F}_{\text{selected}} = \mathcal{F}_{\text{ANOVA}} \cap \mathcal{F}_{\text{RF}} \quad (16)$$

with a fallback to $\mathcal{F}_{\text{ANOVA}} \cup \mathcal{F}_{\text{RF}}$ (truncated to k) if the intersection contains fewer than $k/2$ features. This intersection strategy ensures that selected features are both statistically significant (passing the F-test) and practically informative (ranked important by the Random Forest), filtering out features that score highly on only one criterion.

The resulting 50 selected features are listed in the meta-data and include all four mathematical model scores, confirming their contribution to classification performance. Fig. 6 illustrates the Venn diagram of the dual selection process. Fig. 7 presents the top-20 features by importance.



Figure 6—Venn diagram of the dual feature selection strategy showing ANOVA F-test and Random Forest importance set intersection.



Figure 7—Top-20 feature importances ranked by Random Forest Gini importance, with mathematical model features highlighted.

paper_figures/fig9_feature_pipeline.png

Figure 8—Complete feature engineering pipeline: from raw URL to 50-dimensional standardized feature vector.

3.3.5. Feature Standardization

After selection, all features are standardized using z-score normalization:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j} \quad (17)$$

where μ_j and σ_j are the training-set mean and standard deviation for feature j . The scaler parameters are persisted and applied identically at inference time.

3.4. Anomaly Detection

Supervised classifiers are inherently limited to detecting phishing patterns represented in their training data. To address zero-day phishing URLs that exhibit fundamentally novel characteristics, the framework incorporates an Isolation Forest Liu et al. (2008) anomaly detector trained on the legitimate URL distribution.

The Isolation Forest operates by recursively partitioning the feature space through random axis-aligned splits. Anomalous points—those with unusual feature combinations—require fewer partitions to isolate, yielding shorter average path lengths in the ensemble of isolation trees. The anomaly score for a sample $\mathbf{x}$ is:

$$s(\mathbf{x}, n) = 2^{-\frac{E[h(\mathbf{x})]}{c(n)}} \quad (18)$$

where $E[h(\mathbf{x})]$ is the expected path length across all trees, $c(n) = 2H(n-1) - 2(n-1)/n$ is the average path length of an unsuccessful search in a binary search tree of n samples, and $H(\cdot)$ is the harmonic number.

The detector is configured with 100 isolation trees and a contamination rate of 10%, establishing the decision boundary

at the 90th percentile of the training anomaly score distribution. URLs flagged as anomalous by the Isolation Forest receive elevated attention in the detection pipeline, even when the supervised classifier assigns low phishing probability. Fig. 9 illustrates the anomaly detection boundary.

paper_figures/fig11_anomaly_detection.png

Figure 9—Isolation Forest anomaly detection boundary in the two principal component dimensions, showing normal versus anomalous URL clusters.

4. Experiments and Results

4.1. Dataset Description

The experimental evaluation uses a dataset of 50,000 labeled URLs comprising approximately equal proportions of legitimate and phishing samples. Legitimate URLs are sourced from Alexa Top Sites rankings and curated web directories, while phishing URLs are collected from PhishTank, OpenPhish, and the APWG eCrime Research dataset. The dataset is split 80/20 into training (40,000 URLs) and testing (10,000 URLs) sets using stratified random sampling to preserve class distribution. All random operations use a fixed seed ($r = 42$) for reproducibility.

4.2. Evaluation Metrics

Model performance is assessed using five standard binary classification metrics:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (19)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (20)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (21)$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (22)$$

$$\text{AUC-ROC} = \int_0^1 \text{TPR}(t) d(\text{FPR}(t)) \quad (23)$$

where TP , TN , FP , FN denote true positives, true negatives, false positives, and false negatives respectively. AUC-ROC provides a threshold-independent measure of discriminative ability.

4.3. Cross-Validation Results

All eleven classifiers are evaluated using five-fold stratified cross-validation on the training set, with AUC-ROC as the primary metric. Table 2 summarizes the results.

Table 2: Five-fold cross-validation AUC-ROC results

Classifier	CV Mean AUC	CV Std
Random Forest	0.99963	$\pm 2.51 \times 10^{-5}$
Gradient Boosting	0.99962	$\pm 3.34 \times 10^{-5}$
XGBoost	0.99962	$\pm 3.14 \times 10^{-5}$
LightGBM	0.99959	$\pm 1.83 \times 10^{-5}$
Neural Network (MLP)	0.99927	$\pm 1.69 \times 10^{-4}$
Extra Trees	0.99921	$\pm 3.20 \times 10^{-5}$
SVM	0.99859	$\pm 2.97 \times 10^{-4}$
Logistic Regression	0.99776	$\pm 2.64 \times 10^{-4}$
KNN	0.99629	$\pm 4.92 \times 10^{-4}$
Gaussian Naive Bayes	0.99172	$\pm 5.17 \times 10^{-4}$
Decision Tree	0.98917	$\pm 1.29 \times 10^{-3}$

The top four classifiers (RF, GB, XGB, LGBM) achieve near-identical cross-validation AUC exceeding 0.9995, with standard deviations on the order of 10^{-5} , indicating exceptional stability across data folds. LightGBM exhibits the lowest variance ($\sigma = 1.83 \times 10^{-5}$), while Decision Tree shows the highest variance ($\sigma = 1.29 \times 10^{-3}$), consistent with its known instability. Fig. 10 presents the cross-validation AUC scores with error bars for all classifiers.

paper_figures/fig2_cv_auc_scores.png

Figure 10—Five-fold cross-validation AUC-ROC scores with standard deviation error bars for all eleven classifiers.

4.4. Test Set Performance

Table 3 reports the test set performance metrics for all twelve models.

XGBoost achieves the highest individual accuracy (99.15%) and F1 score (0.9915), marginally outperforming Gradient Boosting (99.10%) and the Neural Network (99.09%). The stacking ensemble achieves 99.07% accuracy with the highest precision (99.33%) among ensemble methods, demonstrating effective aggregation of base learner strengths. All top-six models achieve $\text{AUC-ROC} \geq 0.9991$, indicating near-perfect class separability. Fig. 11 presents the comparative accuracy and F1 scores, and Fig. 12 shows the complete performance heatmap.

Table 3: Test set performance metrics for all models

Model	Accuracy	Precision	Recall	F1 Score	AUC-ROC
XGBoost	0.9915	0.9940	0.9890	0.9915	0.9996
Gradient Boosting	0.9910	0.9928	0.9892	0.9910	0.9996
Neural Network	0.9909	0.9916	0.9902	0.9909	0.9992
Stacking Ensemble	0.9907	0.9933	0.9880	0.9906	0.9996
LightGBM	0.9905	0.9935	0.9874	0.9904	0.9996
Random Forest	0.9900	0.9941	0.9858	0.9899	0.9996
SVM	0.9858	0.9875	0.9839	0.9857	0.9989
KNN	0.9854	0.9897	0.9809	0.9853	0.9963
Extra Trees	0.9829	0.9957	0.9699	0.9826	0.9991
Decision Tree	0.9794	0.9879	0.9705	0.9791	0.9898
Logistic Regression	0.9793	0.9864	0.9719	0.9791	0.9974
Gaussian Naive Bayes	0.9704	0.9851	0.9551	0.9698	0.9917

paper_figures/fig1_model_comparison.png

Figure 11—Comparative accuracy and F1 scores for all twelve models (eleven base classifiers and stacking ensemble).

paper_figures/fig13_performance_heatmap.png

Figure 12—Performance heatmap across all models and evaluation metrics.

4.5. Training Efficiency

Table 4 reports the training time comparison across all classifiers.

Table 4: Training time comparison

Model	Training Time (s)	CV AUC
Gaussian Naive Bayes	0.36	0.9917
Logistic Regression	1.28	0.9978
Decision Tree	1.91	0.9892
LightGBM	3.35	0.9996
XGBoost	4.10	0.9996
KNN	4.87	0.9963
Extra Trees	35.76	0.9992
Random Forest	58.25	0.9996
Neural Network	198.00	0.9993
Gradient Boosting	246.46	0.9996
SVM	397.28	0.9986

LightGBM and XGBoost achieve the best efficiency-performance trade-off, attaining top-tier AUC (0.9996) in under 5 seconds of training time. In contrast, SVM requires 397 seconds—nearly 100× slower than XGBoost—while achieving a lower AUC of 0.9986. Gaussian Naive Bayes trains in 0.36 seconds but suffers a 0.8% AUC penalty relative to XGBoost. The gradient-boosted tree methods (XGB, LGBM) therefore represent the Pareto-optimal classifiers on the accuracy-efficiency frontier. Fig. 13 visualizes the training time comparison.

paper_figures/fig3_training_time.png

Figure 13—Training time comparison across all classifiers (log scale), with AUC annotated.

4.6. Mathematical Model Feature Contribution

To quantify the contribution of the four mathematical models to overall classification performance, we analyze their positioning within the dual feature selection output. All four mathematical features—typosquatting_score, domain_perplexity_score,

homoglyph_score, and domain_entropy_score—are retained in the selected 50-feature subset, confirming their statistical significance (ANOVA F-test) and practical importance (Random Forest Gini ranking).

Table 5: Mathematical feature importance rankings

Mathematical Feature	Description	Selected
typosquatting_score	DL distance to nearest brand	✓
domain_perplexity_score	N-gram LM perplexity	✓
homoglyph_score	Unicode confusable detection	✓
domain_entropy_score	Shannon entropy of domain	✓

The interaction features length_url_x_nb_dots, length_url_x_nb_hyphens, length_url_div_nb_dots, and length_url_div_nb_slash are also all retained, validating the interaction engineering stage.

5. Discussion

5.1. Comparative Analysis

The experimental results demonstrate that PhishGuard achieves state-of-the-art performance using exclusively URL-computable features. The 99.15% accuracy of XGBoost and 99.07% accuracy of the stacking ensemble compare favorably with recent literature: Sahingoz et al. (2019) reported 97.98% with Random Forest on a smaller feature set, Rao and Pais (2019) achieved 98.4% using a voting ensemble with content features, and Aljofey et al. (2020b) reported 97.8% with LSTM networks on raw URL characters.

The near-identical AUC scores (0.9996) achieved by RF, GB, XGB, LGBM, and the stacking ensemble suggest that the feature representation has reached saturation for this dataset, with diminishing returns from classifier diversity beyond the gradient-boosted tree family. The stacking ensemble provides marginal improvement in precision (99.33% vs. 99.40% for RF) but does not surpass XGBoost in accuracy, suggesting that the base learner predictions are highly correlated and offer limited complementary information at this performance level.

5.2. Model Diversity and Ensemble Behavior

The performance stratification reveals three distinct classifier tiers: (i) gradient-boosted trees and neural networks (99.0–99.2% accuracy), (ii) kernel and distance-based methods including SVM and KNN (98.5–98.6%), and (iii) probabilistic and linear models including Naive Bayes and Logistic Regression (97.0–97.9%). This stratification aligns with the complexity of the decision boundaries required for phishing detection: the feature space contains non-linear interactions (captured by trees and neural networks) that linear and independence-assuming models cannot fully exploit.

The Neural Network (MLP with 128-64-32 architecture) achieves the highest recall (99.02%) among all models, suggesting superior sensitivity to minority-class phishing patterns, while Random Forest achieves the highest individual precision (99.41%), indicating fewer false positives. The stacking ensemble effectively balances these complementary strengths.

5.3. Role of Mathematical Features

The retention of all four mathematical features through the dual selection process provides empirical evidence for their discriminative value. Notably:

- **Typosquatting score** captures a class of phishing attacks (brand impersonation through deliberate misspellings) that structural features alone cannot detect, as typosquatting domains may have perfectly normal length, character distribution, and TLD characteristics.
- **N-gram perplexity** provides a principled probabilistic measure of domain naturalness, complementing the cruder heuristic of character entropy by modeling sequential character dependencies.
- **Homoglyph score** detects a qualitatively different attack vector (Unicode substitution) that is invisible to purely ASCII-based feature extraction.
- **Entropy score** correlates strongly with DGA usage, providing a fast screening mechanism for algorithmically generated domains.

5.4. Practical Deployment Considerations

The framework’s exclusive reliance on URL-computable features enables sub-second inference latency suitable for real-time deployment in browser extensions, DNS resolvers, and email gateways. Feature extraction and model prediction complete in under 50 milliseconds on commodity hardware, with no network requests required.

The Isolation Forest anomaly detector addresses the critical cold-start problem: when a new phishing campaign employs a fundamentally novel URL structure absent from training data, the supervised classifiers may assign low phishing probability, but the anomaly detector flags the URL as statistically unusual, enabling escalation for manual review or secondary analysis.

5.5. Limitations

Several limitations should be acknowledged. First, the framework operates exclusively on URL-level features and cannot detect phishing pages that use legitimate-appearing URLs (e.g., compromised legitimate domains). Second, the typosquatting detector is limited to brands in the reference list and will not detect squatting of domains not present in the Alexa Top 1K. Third, the homoglyph detector covers a finite set of known confusable characters and may miss novel Unicode substitution patterns. Fourth, the dataset, while substantial at 50,000 samples, may not capture the full diversity of phishing techniques observed in the wild. Fifth, the fixed perplexity and entropy thresholds are empirically determined and may require recalibration for different linguistic or domain contexts.

6. Conclusion

This paper presented PhishGuard, a comprehensive machine learning framework for phishing website detection that integrates mathematical feature engineering with ensemble classification. The framework introduces four specialized mathematical models—Damerau-Levenshtein typosquatting scoring, character n-gram language modeling, Unicode homoglyph detection, and Shannon entropy analysis—that extract semantic and statistical signals from URL strings beyond what traditional structural features capture.

Evaluated on a dataset of 50,000 URLs, the proposed system achieves 99.15% accuracy with XGBoost and 99.07% accuracy with the stacking ensemble, with AUC-ROC of 0.9996 for the top models. The dual feature selection strategy validates the contribution of all four mathematical features by retaining them through both ANOVA F-test and Random Forest importance filtering. The Isolation Forest anomaly detector provides complementary unsupervised coverage for zero-day phishing patterns.

Future work will explore several directions: (i) expanding the mathematical model suite with TLS certificate analysis and DNS-based features, (ii) implementing online learning for continuous model adaptation to evolving phishing tactics, (iii) investigating adversarial robustness through feature perturbation studies, (iv) extending the framework to multi-class classification distinguishing phishing from spam, malware, and legitimate URLs, and (v) evaluating transfer learning approaches to reduce dataset requirements for new deployment contexts.

Acknowledgments

The author thanks the open-source communities behind scikit-learn, XGBoost, and LightGBM for providing robust machine learning implementations, and the Anti-Phishing Working Group (APWG), PhishTank, and Alexa Internet for the datasets and domain lists used in this research.

References

- Ahmed, M., Mahmood, A.N., Hu, J., 2016. A survey of network anomaly detection techniques. *Journal of Network and Computer Applications* 60, 19–31.
- Aljofey, A., Jiang, Q., Qu, Q., Huang, M., Niyigena, J.P., 2020a. An effective phishing detection model based on character level convolutional neural network from URL. *Electronics* 9, 1514.
- Aljofey, A., Jiang, Q., Qu, Q., Huang, M., Niyigena, J.P., 2020b. An effective phishing detection model based on character level convolutional neural network from URL. *Electronics* 9, 1514.
- Anti-Phishing Working Group, 2024. Phishing Activity Trends Report, 4th Quarter 2023. Tech. Rep.. APWG.

- Buber, E., Diri, B., Sahingoz, O.K., 2017. NLP based phishing attack detection from URLs, in: Proceedings of the International AI and Data Processing Symposium (IDAP), pp. 1–5.
- Chandola, V., Banerjee, A., Kumar, V., 2009. Anomaly detection: A survey. *ACM Computing Surveys* 41, 1–58.
- Chen, T., Guestrin, C., 2016. XGBoost: A scalable tree boosting system, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794.
- Chiew, K.L., Yong, K.S.C., Tan, C.L., 2018. A survey of phishing attacks: Their types, vectors and technical approaches. *Expert Systems with Applications* 106, 1–20.
- Chou, N., Ledesma, R., Teraguchi, Y., Boneh, D., Mitchell, J.C., 2004. Client-side defense against web-based identity theft, in: Proceedings of the Network and Distributed System Security Symposium (NDSS).
- Damerau, F.J., 1964. A technique for computer detection and correction of spelling errors. *Communications of the ACM* 7, 171–176.
- Google, 2024. Google Safe Browsing. <https://safebrowsing.google.com>.
- Holgers, T., Watson, D.E., Gribble, S.D., 2006. Cutting through the confusion: A measurement study of homograph attacks, in: Proceedings of the Annual Technical Conference (ATC), pp. 261–266.
- Invernizzi, A., Thomas, K., Kapravelos, A., Comanescu, O., Picod, J.M., Bursztein, E., 2016. Cloak of visibility: Detecting when machines browse a different web, in: Proceedings of the IEEE Symposium on Security and Privacy (SP), pp. 743–758.
- Jain, A.K., Gupta, B.B., 2022. A survey of phishing attack techniques, defence mechanisms and open research challenges. *Enterprise Information Systems* 16, 527–565.
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.Y., 2017. LightGBM: A highly efficient gradient boosting decision tree, in: Proceedings of Advances in Neural Information Processing Systems (NeurIPS), pp. 3149–3157.
- Khonji, M., Iraqi, Y., Jones, A., 2013. Phishing detection: A literature survey. *IEEE Communications Surveys & Tutorials* 15, 2091–2121.
- Le, H., Pham, Q., Sahoo, D., Hoi, S.C.H., 2018a. URLNet: Learning a URL representation with deep learning for malicious URL detection. *arXiv preprint arXiv:1802.03162*.
- Le, H., Pham, Q., Sahoo, D., Hoi, S.C.H., 2018b. URLNet: Learning a URL representation with deep learning for malicious URL detection. *arXiv preprint arXiv:1802.03162*.
- Liu, F.T., Ting, K.M., Zhou, Z.H., 2008. Isolation forest, in: Proceedings of the IEEE International Conference on Data Mining (ICDM), pp. 413–422.
- Ma, J., Saul, L.K., Savage, S., Voelker, G.M., 2009a. Beyond blacklists: Learning to detect malicious web sites from suspicious URLs, in: Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1245–1254.
- Ma, L., Ofoghi, B., Watters, P., Brown, S., 2009b. Detecting phishing emails using hybrid features, in: Proceedings of the Symposium on Cybercrime and Trustworthy Computing, pp. 19–26.
- Marchal, S., François, J., State, R., Engel, T., 2014. Phish-Storm: Detecting phishing with streaming analytics. *IEEE Transactions on Network and Service Management* 11, 458–471.
- Mohammad, R.M., Thabtah, F., McCluskey, L., 2015. Phishing websites features. Tech. Rep.. School of Computing, University of Huddersfield.
- Muhammad, A.I., Orunsolu, A.R., Akinwale, A.T., 2023. Feature-based phishing detection system using machine learning techniques. *Neural Computing and Applications* 35, 1–18.
- PhishTank, 2024. PhishTank: Join the fight against phishing. <https://phishtank.org>.
- Rao, R.S., Pais, A.R., 2019. Detection of phishing websites using an efficient feature-based machine learning framework. *Neural Computing and Applications* 31, 3851–3873.
- Sahingoz, O.K., Buber, E., Demir, O., Diri, B., 2019. Machine learning based phishing detection from URLs. *Expert Systems with Applications* 117, 345–357.
- Sahoo, D., Liu, C., Hoi, S.C.H., 2017. Malicious URL detection using machine learning: A survey. *arXiv preprint arXiv:1701.07179*.
- Shannon, C.E., 1948. A mathematical theory of communication. *Bell System Technical Journal* 27, 379–423.
- Shwartz-Ziv, R., Armon, A., 2022. Tabular data: Deep learning is not all you need. *Information Fusion* 81, 84–90.
- Szurdi, J., Kocso, B., Cseh, G., Spring, J., Felegyhazi, M., Kanich, C., 2014. The long tail of typosquatting domain names, in: Proceedings of the 23rd USENIX Security Symposium, pp. 191–206.
- Verizon, 2024. 2024 Data Breach Investigations Report. Tech. Rep.. Verizon Enterprise Solutions.
- Verma, R., Dyer, K., 2015. On the character of phishing URLs: Accurate and robust statistical learning classifiers, in: Proceedings of the 5th ACM Conference on Data and Application Security and Privacy, pp. 111–122.

Wolpert, D.H., 1992. Stacked generalization. *Neural Networks* 5, 241–259.

Xiang, G., Hong, J., Rose, C.P., Cranor, L., 2011. CANTINA+: A feature-rich machine learning framework for detecting phishing web sites. *ACM Transactions on Information and System Security* 14, 1–28.

Zhang, Y., Hong, J.I., Cranor, L.F., 2007. CANTINA: A content-based approach to detecting phishing web sites, in: *Proceedings of the 16th International Conference on World Wide Web (WWW)*, pp. 639–648.